INKRA

PX-875 Design Specification

Timeline Impact Calculation
Rule Engine

Ticket	Status	Type	Depends On
PX-875	Backlog / Future	Spec — No Code	PX-882, Goal/OKR system

1. Overview

When a conversation reveals that work is running behind — a developer says "I need an extra week for testing," or a case manager says "I'm blocked until legal signs off" — Inkra should do more than log an action item. It should trace that delay forward through dependent tasks, up into parent programs and goals, and express the impact as a concrete timeline shift and a percentage threat to any linked OKR or organizational objective.

This document specifies the rule engine that performs that calculation. It is a future-state design document. No implementation work is required before the Goal/OKR tracking system is built and PX-882 (the workflow router) is live.

> **Design Philosophy**
> The engine is deterministic by default — given the same inputs it always produces the same outputs. ML is used only for classifying ambiguous natural language, not for the math itself. Every impact number must be explainable: when the system says "this puts your Q2 goal at 23% risk," it must show exactly how it arrived there. No magic numbers.

What the Engine Receives

- **Delay signal:** extracted utterance, classified tier (T1–T7), stated duration, and NLP confidence score
- **Task context:** affected task ID, owner, start date, original due date, current progress %, linked sprint if applicable
- **Goal/OKR graph:** linkage tree: task → parent program/project → parent goal, with weights and deadlines at each level
- **Dependency graph:** all tasks that are blocked by or that block the affected task

What the Engine Produces

- Revised completion date for the directly affected task
- Cascading slip for each downstream dependent task on the critical path
- Revised completion date and health-impact percentage for each linked goal/OKR
- Human-readable summary for the Review UI
- requires_human_review flag + reason when calculation confidence is too low to auto-apply

2. Delay Signal Classification (T1–T7)

Before any calculation can happen, the NLP extraction layer must classify the delay signal into one of seven tiers. Each tier drives a different calculation path.

Tier	Name	Example Utterances	Confidence	Duration Calc	Action	Auto-Apply
T1	Precise	"I need exactly 3 more days" / "Done by Friday" / "Pushes to March 22"	0.95	Use verbatim value	Recalculate + update	Yes
T2	Ranged	"Two or three more days" / "A couple weeks" / "A week, maybe more"	0.80	Midpoint x 0.80	Recalculate + notify	Yes
T3	Vague-Certain	"Going to take longer" / "Won't make the deadline"	0.50	Prompt for duration	Human review	No
T4	Vague-Uncertain	"Might take a bit more" / "Could be delayed"	0.30	Risk flag only	Human review	No
T5	Blocked	"Blocked on X" / "Can't proceed until Y" / "Waiting on legal"	N/A	Estimate from type avg	Create blocker; escalate	No
T6	Scope Change	"We need to add X" / "Client changed specs"	0.60	Route to estimation	Route to PM	No
T7	Completion	"Finished X" / "Testing is done" / "That's wrapped up"	0.90	Remove remaining work	Recalculate positive	Yes

2.1 Duration Extraction — Phrase-to-Day Mapping

The NLP layer maps common phrasings to day counts before handing off to the rule engine. For ranges, the midpoint is used.

Phrase Pattern	Min Days	Max Days	Midpoint	Notes
"a day" / "one day"	1	1	1	
"a couple days" / "a few days"	2	3	2.5	
"several days"	3	5	4	
"about a week" / "a week or so"	5	7	6	

Phrase Pattern	Min Days	Max Days	Midpoint	Notes
"a week and a half"	8	10	9	
"two weeks" / "a couple weeks"	10	14	12	
"a month"	20	22	21	Working days approximation
"an extra sprint"	—	—	sprint_length	Pull org sprint_length config; default 14
"by [day name]" e.g. "by Friday"	—	—	Computed	today to target weekday, business days
"by [date]" e.g. "by March 22"	—	—	Computed	today to exact date, business days only

2.2 Confidence Discounting for T2 (Ranged) Signals

Speakers systematically underestimate delays. For ranged signals, the engine applies a 0.80 factor to the midpoint:

```
// T2 effective delay
stated_delay_days   = (range_min + range_max) / 2
effective_delay_days = stated_delay_days * 0.80

// Example: "two or three more days"
stated   = (2 + 3) / 2 = 2.5
effective = 2.5 * 0.80 = 2.0  // forwarded to Section 3
```

Phase 2 enhancement: once the system has 10 or more resolved delays per team, it calibrates the confidence factor against historical actuals. If a team's estimates run 30% low, the factor is updated to 1.30.

3. Core Impact Calculation

3.1 Task-Level Calculation

Step one: compute the revised completion date for the directly affected task.

```
INPUTS
  task.original_due_date        // promised delivery date
  task.progress_pct             // 0.0-1.0 from PM system or estimated
  task.original_duration_days   // due_date - start_date in business days
  effective_delay_days          // from Section 2
  today                         // date of conversation

CALCULATION
  remaining_work_days  = task.original_duration_days * (1.0 -
task.progress_pct)
  new_completion_date  = today + remaining_work_days + effective_delay_days
  slip_days            = max(0, new_completion_date -
task.original_due_date)
  task_impact_pct      = (slip_days / task.original_duration_days) * 100

  // T7 (completion): slip_days negative (ahead of schedule)
  // max(0,...) for display only; raw negative value preserved internally
```

Worked Example — T1 Precise Delay

Utterance: "I need an extra week for testing"
Task: API integration testing
Start: Mar 1 | Due: Mar 15 | Progress: 60% | Today: Mar 10

remaining_work_days = 14 * (1 - 0.60) = 5.6 days
effective_delay_days = 7 (T1, confidence 1.0)
new_completion_date = Mar 10 + 5.6 + 7 = Mar 23
slip_days = Mar 23 - Mar 15 = 8 days
task_impact_pct = (8 / 14) * 100 = 57%

3.2 Critical Path and Float Analysis

Not every slipping task drives a goal deadline slip. Before cascading, the engine checks whether the task has enough float to absorb the delay.

```
// For each downstream task D that depends on affected task T:
float_days[D] = D.latest_allowable_start - D.earliest_allowable_start
```

```
if slip_days <= float_days[D]:
  // D is NOT on the critical path for this slip
  D.float_remaining = float_days[D] - slip_days
  // record float consumption; do NOT cascade further

else:
  // D IS on the critical path — cascade the overflow
  D.cascaded_slip    = slip_days - float_days[D]
  D.new_start_date   = D.original_start_date + D.cascaded_slip
  D.new_due_date     = D.original_due_date   + D.cascaded_slip
  // recurse: apply D's slip to D's own dependents
```

The engine traverses the full dependency graph breadth-first, stopping propagation at any node where remaining float absorbs the incoming slip.

3.3 Goal / OKR Impact Calculation

Once task-level and cascade slips are resolved, the engine calculates the effect on any linked Goal or OKR. Each goal has a deadline and one or more contributing tasks with weights.

```
// Goal health: 1.0 = fully on track
// Each task T has weight W (default: 1/N for N tasks)

// Step 1: per-task health
task_health[T] = 1.0 - min(1.0, slip_days[T] / task.original_duration_days)

// Step 2: weighted goal health
goal_health_new = SUM(task_health[T] * weight[T])  for all T in goal.tasks
goal_health_old = previous snapshot (1.0 if first run)

// Step 3: risk delta
goal_risk_pct = (goal_health_old - goal_health_new) * 100

// Step 4: revised deadline estimate
goal_new_deadline = MAX(task.new_completion_date for T in goal.tasks
                        where T is on critical path for goal)

// Step 5: goal-level slip and buffer
goal_slip_days      = max(0, goal_new_deadline - goal.original_deadline)
buffer_remaining    = goal.original_deadline - goal_new_deadline
  // positive = buffer still exists; negative = deadline breach
```

Worked Example — Goal Impact

Goal: Ship v1.0 by March 31 (3 tasks, equal weight 0.33 each)

```
Task A:  API integration   — slips 8 days
Task B:  Frontend UI       — on track (0 slip)
Task C:  QA / load test    — depends on A; cascaded slip = 5 days (float absorbed 3)

task_health[A] = 1.0 - min(1.0, 8/14)  = 0.43
task_health[B] = 1.0                   = 1.00
task_health[C] = 1.0 - min(1.0, 5/10)  = 0.50

goal_health_new = (0.43*0.33) + (1.00*0.33) + (0.50*0.33) = 0.64
goal_risk_pct   = (1.0 - 0.64) * 100 = 36%

goal_new_deadline = MAX(Mar 23, Mar 15, Mar 28) = Mar 28
goal_slip_days    = max(0, Mar 28 - Mar 31) = 0  (3-day buffer remains)

Summary: "v1.0 goal is 36% at risk. Estimated delivery Mar 28 (3 days buffer remaining)."
```

4. Blocker Handling (T5)

Blocked tasks have no stated duration. The engine cannot calculate a precise delay until the blocker resolves. It applies tiered estimation based on blocker type.

4.1 Blocker Estimation Tiers

Blocker Type	Detection Signal	Default Est.	Escalation At	Resolution Trigger
Internal-team	"Waiting on [teammate]" / "Blocked by [person]"	2 biz days	2 days	Teammate confirms completion in conversation
Cross-team	"Waiting on [other team]" / "Needs [group] to..."	5 biz days	3 days	Cross-team update mentions unblock
External / vendor	"Waiting on legal" / "Waiting on client"	10 biz days	5 days	External party acknowledged; manual close
Technical / env	"Environment is down" / "CI is broken"	1 biz day	Same day if infra	Automated: monitor detects env restored
Approval / review	"Waiting for sign-off" / "Needs approval"	3 biz days	2 days	Approver mentions decision in subsequent conversation

4.2 Blocker Workflow

```
on T5 signal detected:
  1. Create blocker ticket linked to affected task
  2. Set task.status = BLOCKED
  3. Apply default_estimated_delay from table above
  4. Run goal impact calculation using estimated delay (flagged is_estimated
= true)
  5. Notify task owner + team lead
  6. Schedule escalation at escalation_threshold

on blocker resolved (detected via subsequent conversation or manual close):
  1. Record actual_blocked_days = resolution_date - block_start_date
  2. Recalculate all impacts using actual_blocked_days
  3. Update downstream tasks and goals
  4. Store in org blocker history (feeds Phase 2 calibration)
```

5. Ambiguity Resolution and Human Review

5.1 When Human Review Is Required

The engine sets requires_human_review = true and pauses auto-application whenever any of the following conditions are met:

Condition	Threshold	Review Prompt
NLP confidence below threshold	< 0.70	Show utterance; ask user to confirm task, duration, and scope
T3 or T4 signal (vague)	Any T3/T4	"How much additional time is needed?" + quick-pick options
Task not resolvable to specific ticket	Entity match < 0.80	Show candidate matches; ask user to select or create new
Calculated goal impact is high	goal_risk_pct > 50%	High-stakes impact; require explicit confirmation before committing
Goal deadline already passed	goal.deadline < today	Goal in breach; do not auto-update; route to manager
Multiple tasks match utterance	match_count > 1	Show list; ask user which task(s) are affected
Scope change signal	Any T6	Route to PM/project owner for effort estimation
Contradicts recent conversation	Conflict detected	Show both utterances; ask which is current

5.2 Review UI Requirements

When human review is required, the Review UI (PX-881) must surface:

- The original utterance with the detected entity highlighted
- The proposed task match with confidence score and match rationale
- The calculated impact preview (task slip, cascade, goal health delta)
- Quick-accept, quick-edit, and dismiss actions
- An expandable "How was this calculated?" trace showing every intermediate value

5.3 Tense and Hypothetical Suppression

The NLP layer must filter out non-actionable delay signals before handing off to the rule engine.

Suppress: Past Tense

"We should have started earlier" — regret, not a current delay
"That took us an extra week" — historical, already accounted for
"Last sprint we were two days late" — retrospective

Detection: past-tense verb forms; no forward-looking date reference.

Suppress: Hypothetical / Conditional

"If the API changes, we might need more time" — not yet triggered
"This could slip if X doesn't happen" — conditional on external event
"In the worst case, two more weeks" — hypothetical scenario

Detection: conditional markers (if, could, might, worst case, hypothetically, assuming X).

6. Data Model

6.1 DelaySignal

```
DelaySignal {
  id                  UUID
  conversation_id     FK → Conversation
  source_utterance    string         // raw extracted text
  tier                enum T1-T7
  stated_min_days     float | null   // null for T3, T4, T5
  stated_max_days     float | null
  effective_days      float | null   // post-discount; used in calculation
  nlp_confidence      float          // 0.0-1.0
  tense               enum past | present | future
  is_hypothetical     boolean
  extracted_at        timestamp
  resolved_task_id    FK → Task | null
  resolution_method   enum auto | human | suppressed
}
```

6.2 TimelineImpactRecord

```
TimelineImpactRecord {
  id                      UUID
  delay_signal_id         FK → DelaySignal
  affected_task_id        FK → Task
  slip_days               float
  new_completion_date     date
  task_impact_pct         float
  is_estimated            boolean        // true if based on T5 default
  requires_human_review   boolean
  review_reason           string | null
  applied_at              timestamp | null
  applied_by              enum auto | user_id
  cascades                CascadeRecord[]
  goal_impacts            GoalImpactRecord[]
}
```

6.3 CascadeRecord

```
CascadeRecord {
  task_id                FK → Task
  float_consumed_days    float
  cascaded_slip_days     float    // 0 if float absorbed the slip
  new_start_date         date | null
```

```
  new_completion_date   date | null
  is_on_critical_path   boolean
}
```

6.4 GoalImpactRecord

```
GoalImpactRecord {
  goal_id                    FK → Goal
  health_before              float
  health_after               float
  goal_risk_pct              float
  new_deadline_estimate      date | null
  goal_slip_days             float
  buffer_remaining_days      float
  contributing_task_impacts  [{ task_id, weight, task_health }]
}
```

6.5 BlockerRecord

```
BlockerRecord {
  id                       UUID
  blocked_task_id          FK → Task
  blocker_type             enum internal | cross_team | external | technical
| approval
  blocker_description      string
  blocked_at               timestamp
  estimated_delay_days     float
  escalation_threshold_at  timestamp
  resolved_at              timestamp | null
  actual_blocked_days      float | null
  resolution_signal        string | null
}
```

7. Edge Case Documentation

Edge Case	Scenario	Engine Behavior
Task already past due	task.due_date < today at time of conversation	Compute slip_days from today, not due_date. Flag task OVERDUE. Do not apply negative slip to future tasks — overdue time is already lost.
Delay exceeds remaining duration	effective_delay_days > remaining_work_days	task_impact_pct capped at 100%. Cascade the full delay. Flag for human review to confirm task is not being abandoned.
Multiple delays in same conversation	Two speakers each state a delay for different tasks	Create independent DelaySignal records. Process serially. If both affect the same goal, compute combined impact across both signals.
Same task delayed multiple times	Task T was already slipped last week; slipping again today	new_completion_date computed from current state, not original. slip_days = cumulative drift from original due date. Each signal stored independently for audit trail.
No progress data available	task.progress_pct is null or 0	Default to 0% (pessimistic). Flag output as estimated. Prompt user in Review UI to confirm or update progress before applying.
Task with no goal linkage	Task exists but has no parent goal/OKR	Compute task-level and cascade impacts only. Log warning that goal linkage is missing; suggest attaching to a goal.
Contradictory signals in same session	"We'll finish this sprint" followed by "I need two more weeks"	Flag contradiction. Set requires_human_review = true. Surface both utterances; ask user which is current.
Blocker resolved in same session	Blocked at minute 5, unblocked at minute 23	Net delay = zero if same-session. Record both signals. Do not propagate to goal if actual_blocked_days < float_days.
T7 completion on early-finish task	Task finishes ahead of schedule	slip_days is negative. Float returned to downstream tasks. Recompute goal health upward. Display 'ahead by X days' not negative slip.
Goal with no deadline	OKR goal exists but deadline is null	Skip goal_slip_days calculation. Still compute goal_risk_pct from task health. Flag that deadline is needed for timeline impact to be actionable.
Circular dependency in task graph	A depends on B, B depends on A	Detect cycle during traversal. Do not recurse. Flag as data quality error. Route to PM before impact can be computed.

Edge Case	Scenario	Engine Behavior
Sprint-relative delay, no config	"An extra sprint" stated but sprint_length not configured	Default to 14 days. Log that sprint_length config is missing. Prompt admin to set it in org settings.

8. Goal / OKR Integration Spec

8.1 Required Goal Fields

- **goal.id:** unique identifier
- **goal.title:** human-readable name
- **goal.deadline:** date — required for timeline impact; nullable but triggers warning
- **goal.contributing_tasks:** array of task IDs with optional weight (0.0–1.0, summing to 1.0)
- **goal.health_score:** float 0.0–1.0, current snapshot; engine reads and writes this field
- **goal.parent_goal_id:** nullable FK for nested OKR hierarchies (Key Result → Objective)

8.2 Weight Assignment Rules

Task weights determine how much each task's slip affects the goal health score. Assignment logic, in priority order:

- Explicit weight: PM has manually set a weight for this task — use as-is
- Effort-based: if tasks have story points or hour estimates, weight = task_effort / total_goal_effort
- Equal weight (default): weight = 1 / N where N = number of contributing tasks

8.3 Nested Goal Rollup

For two-tier OKR structures (Key Results roll up into Objectives), the engine applies goal impact recursively:

```
objective_health = SUM(KR.health_score * KR.weight_in_objective)
objective_risk_pct = (objective_health_old - objective_health_new) * 100

// Maximum nesting depth: 3 levels (Task → Key Result → Objective)
// Beyond 3 levels: flag as unsupported; do not cascade further
```

8.4 Notification Thresholds

Threshold	Default	Notify	Channel
goal_risk_pct crosses 10%	> 10%	Goal owner	In-app notification
goal_risk_pct crosses 25%	> 25%	Goal owner + team lead	In-app + email digest
goal_risk_pct crosses 50%	> 50%	Goal owner + team lead + stakeholder	In-app + email + Slack

Threshold	Default	Notify	Channel
goal_slip_days > 0 (deadline breach)	Any breach	All above + exec summary	All channels; human review required before commit
Buffer remaining < 3 days	< 3 days	Goal owner + PM	In-app + Slack

9. Determinism, Auditability, and ML Boundary

9.1 Rule vs. ML Boundary

A clear boundary separates ML-assisted classification from deterministic rule-based calculation.

ML-Assisted (NLP Layer)	Rule-Based (This Engine)
Delay signal detection in transcript	Duration extraction and arithmetic
Tier classification (T1–T7)	Confidence discounting formula
Duration phrase parsing ("a couple weeks")	Cascade traversal through dependency graph
Task entity resolution (which task is being discussed)	Float and critical path analysis
Tense and hypothetical detection	Goal health score calculation
Confidence scoring	Notification threshold evaluation
Blocker type classification	All threshold comparisons and flag logic

9.2 Auditability Requirements

Every TimelineImpactRecord must be fully reproducible. Store with each record:

- Source utterance verbatim
- Tier classification and confidence score
- Every intermediate value used in the calculation
- Snapshot of task.progress_pct and task.original_due_date at time of calculation
- User ID and timestamp of any human review action
- Whether the record was auto-applied or human-confirmed

The "How was this calculated?" expansion in the Review UI must render from stored values — not re-run the formula live. This ensures the explanation always matches what actually ran.

9.3 Idempotency

If the same conversation is processed twice (e.g., retry), the engine must not create duplicate records. Detection: hash on (conversation_id, source_utterance, extracted_at). If a matching record exists, return it rather than creating a new one.

10. Implementation Notes and Open Questions

10.1 Progress Percentage Sourcing

The engine requires task.progress_pct to compute remaining work. Sourcing strategy in priority order:

- PM system field: if Jira/Linear/Asana exposes a completion percentage, use it directly
- Completed sub-tasks ratio: completed child issues / total child issues
- Story points: completed points / total points for the task
- Default fallback: 0% (pessimistic) — flag as estimated and surface in Review UI

A config option should let orgs override the default fallback per project (e.g., "assume 50%" for teams that rarely update progress fields).

10.2 Business Days Configuration

All duration calculations use business days by default. The engine must respect:

- Organization-configured work week (default Monday–Friday)
- Organization-configured holidays (defaults to US federal holidays if unset)
- Time zone of the task owner for date boundary calculations

10.3 Open Questions for Phase 1

Question	Options	Recommended
How is task weight set on a goal?	Manual / Effort-based / Equal default	Equal default; surface weight editor in goal UI
Minimum confidence for T1 auto-apply?	0.80 / 0.85 / 0.90 / configurable	0.85; configurable per org
Should T7 auto-update goal health immediately?	Yes / No / With confirmation	Yes for < 25% impact; confirm for > 25%
How long are DelaySignal records retained?	Forever / 1 year / configurable	1 year default; configurable for compliance
Does engine run per-message or end-of-conversation?	Per message / End-of-conversation batch	End-of-conversation batch for v1; real-time in v2
How are sprint_length and holidays configured?	Org-level / Per-project / Per-team	Org-level settings with per-team override

10.4 Phase 2 Enhancements

- Historical calibration of confidence factors per team (Section 2.2)
- Real-time delay detection during live recording (v1 is post-conversation batch only)
- Predictive risk scoring: flag tasks likely to slip based on patterns before a delay is stated
- Cross-conversation trend analysis: flag teams whose estimates run chronically low
- OKR alignment scoring: surface which conversations contribute most to goal progress vs. off-mission work